

Le Shoe Shop — Ops Dashboard: integration brief

For whoever (or whichever AI) is wiring this into leshoeshop.ca. This doc is self-contained — you shouldn't need anything else from this project to build against it.

Context

Le Shoe Shop is a sneaker/shoe cleaning business (pickup + delivery, Québec City area). A separate project (this repo) built out the backend data model and a static design concept for an internal operations dashboard. That work is done; **this brief is the task**: build the real dashboard on leshoeshop.ca's existing site/admin area, wired to live data instead of the mockup's placeholder rows.

Scope for this integration — orders, inventory, deliveries, and a deliverer-facing view.

Sponsorship (a separate Lions Basketball cross-venture feature) and owner reimbursement/expense tracking exist in the schema but are explicitly **out of scope** for now — not essential yet, leave them alone. Don't build UI for `sponsorship_deal` , `sponsorship_deliverable` , `sponsorship_reach_snapshot` , `game` , `expense` , or `owner_loan` .

Mapping onto the existing admin nav

The admin already has: Dashboard · Bookings · Clients · Calendrier · Produits · Commandes · Driver · Galerie. This project's models map onto those tabs as:

Tab	Model(s)
Bookings	<code>Order</code> / <code>OrderItem</code> / <code>Payment</code> — the cleaning service pipeline (Refresh/Reset/Renew)
Commandes	<code>ProductOrder</code> / <code>ProductOrderItem</code> — standalone product sales (buying a cleaner/pack with no service booked). Deliberately a separate model from <code>Order</code> , not a variant of it — see below.
Clients	<code>Customer</code>
Produits	<code>Product</code> (the boutique catalog) — and probably where par-level inventory (<code>StockItem</code>) belongs too, even though those are conceptually different (sellable products vs. internal consumables)
Driver	The deliverer-facing view — see its own section below
Calendrier	<code>AvailabilitySlot</code> — weekly recurring pickup availability, see below
Galerie	Before/after photos — simple enough that it doesn't need a model from this project; build it however fits the rest of the admin (e.g. a Storage bucket + a simple upload UI)

Tab	Model(s)
Dashboard	The ops overview from this brief — orders pipeline + inventory par levels + deliveries

Design reference

Static concept: `Main.dc.html` in this folder, also viewable at <https://claude.ai/code/artifact/dbe0d82e-ef93-431b-995c-b35e255470d2> (placeholder data — layout/hierarchy is the point, not the sample rows).

Three sections, in priority order: **Orders** (pipeline by status: pending → deposit paid → in progress → cleaned → delivered → paid, tier badges for Refresh/Reset/Renew), **Inventory** (par-level bars per stock item, flag anything at/below par), **Deliveries** (upcoming pickups/ drop-offs). No dollar amounts anywhere in this dashboard — deliverers/ employees may get access to it, and pricing isn't something they need to see.

Type: Space Grotesk (headers) + IBM Plex Sans (body), Google Fonts. Warm amber accent, semantic red/amber/green for status. No leshoesshop.ca design system was available when this was drafted — **use the site's real tokens instead of these if they conflict**; this palette was a placeholder, not a mandate.

Data source

Supabase project `le-shoe-shop` (ref `yvymhjtswcneouvvgfbc`).

- REST URL: `https://yvymhjtswcneouvvgfbc.supabase.co/rest/v1/`
- Anon (publishable) key — safe for client-side use, RLS already scopes what it can read:
`sb_publishable_TFJxA1R5m5aGALAWZKLpRQ_bPNg9MWK`
- Full schema: `prisma/schema.prisma` in the source repo. Table names are snake_case (`order` , `stock_item` , `customer` , `order_item` , `subcontractor` , `shop_location`) — columns are listed in the migrations under `supabase/migrations/` .

RLS currently grants public **read** on `customer` , `order` (restricted columns — see below), `order_item` , `stock_item` , `subcontractor` , `shop_location` . Nothing else is reachable with this key (financials, expenses, sponsorship, invoices are service-role-only, and out of scope here anyway).

order 's dollar columns are blocked at the database level for the anon key, not just hidden by convention — `subtotal` , `tps` , `tvq` , `total` , `pickup_fee` , `volume_discount_pct` will error with "permission denied" if queried with this key. `product_order` has the same treatment (`total` / `subtotal` / `tps` / `tvq` blocked, `status` /dates readable). This is deliberate (see Deliverer view below) — don't try to work around it by adding a view that re-exposes them to anon; if the owner-facing dashboard needs totals, that's a service-role/authenticated concern.

This whole anon-key access model is a placeholder, not a design to extend. It exists only because no real auth is built yet. Once you build real authentication/authorization for the admin, replace this entirely rather than adding more anon-key column grants for new tables — treat every grant currently on `anon` as scaffolding to remove, not a pattern to follow.

No auth exists yet. This was fine to open up because `customer` / `order` are still empty at time of writing (0 rows). Before real customer data goes live: either add row-level scoping (e.g. per logged-in staff user) or move `customer` fields with personal info (email/phone/address) behind an authenticated read instead of the public anon key. Flag this back to the source repo if you need help deciding — it's a schema/access decision, not a frontend one.

Queries per section

Orders — group client-side by `status` :

```
GET /order?select=*,customer(name,address)&order=booked_at.desc
```

`tier` is `REFRESH` | `RESET` | `RENEW`. `status` is `PENDING` | `DEPOSIT_PAID` | `IN_PROGRESS` | `CLEANED` | `DELIVERED` | `PAID` | `CANCELLED`. `order.notes` holds delivery specifications/instructions (e.g. "leave at door", "call before arriving") — surface it wherever a deliverer sees the order.

Mid-order add-ons and upgrades — already supported structurally, no schema change needed: - Add-on (extra laces, crease protectors, etc.) → insert a row into `order_item` (`type` = 'ADDON' , `code` , `quantity` , `price`) at any time, even after the order was originally booked. - Tier upgrade (Refresh → Reset → Renew) → update `order.tier` , then recompute and update `subtotal` / `tps` / `tvq` / `total` to match the new tier's price. That recompute is application logic — there's no DB trigger doing it automatically, so whichever screen handles "upgrade this order" needs to do the math and write all four fields together.

Commandes (product sales, separate from Bookings) —

```
GET /product_order?select=*,customer(name),product_order_item(*,product(name,sku))&order=ordered_at.
```

`status` is `PENDING` | `PAID` | `FULFILLED` | `CANCELLED`. This is a **separate model from `Order`** on purpose — a product-only purchase has no `ServiceTier` , no pair count, no pickup/delivery. Don't try to represent it as an `Order` with nulled-out fields.

Inventory —

```
GET /stock_item?select=*
```

Flag `current_qty <= par_level` for the at/below-par warning. **This table is currently empty** — someone needs to enter real par levels before this section shows anything. Laces specifically: track **both round and flat styles**, colors limited to **white and black only** (so four SKUs/rows if tracked separately: round-white, round-black, flat-white, flat-black — or two rows if style isn't worth splitting and only color is). Other items: cleaning solution, deodorizer, crease protectors, waterproof spray, brushes. Not a frontend problem — ping the source repo owner to seed it, or seed it yourselves if you have the numbers.

Deliveries / pickup-fee distance —

```
GET /shop_location?venture=eq.LE_SHOE_SHOP&select=*
```

Returns the shop's canonical address (**11 rue Langevin**) and lat/lng if geocoded. **The pickup/delivery fee must be calculated as distance FROM this address to the customer's address** — whatever the current booking flow uses instead is producing skewed fees and needs to be replaced with a real distance calculation (e.g. Google Maps Distance Matrix API) anchored on this row, not a hardcoded or wrong origin point. This was flagged as a real, currently-wrong calculation — treat it as a bug fix, not a nice-to-have.

No dedicated delivery/route/zone model exists in the schema yet. The mockup invented zone grouping (Sainte-Foy, Lévis, etc.) from `customer.address` text for illustration only — there is no real `zone` column to query. If zone-grouped deliveries are wanted for real, that's a schema addition, not something to fake client-side. Flag it back rather than guessing a parsing scheme.

Calendrier — weekly recurring availability

This replaces whatever date-based calendar exists now. Pickup availability is a repeating weekly pattern (e.g. "Tuesdays 2-5pm, Thursdays 10am-1pm"), not a set of specific calendar dates — set it once and it applies every week going forward, no re-entering dates.

```
GET /availability_slot?venture=eq.LE_SHOE_SHOP&active=eq.true&select=*
```

`weekday` is `MONDAY ... SUNDAY`, `start_time / end_time` are `"HH:MM"` 24h strings. The booking UI resolves a customer's pick (a specific weekday + time within an active slot) into an actual calendar date for that particular week, and writes it to `order.pickup_scheduled_at` (a real timestamp, since one specific order happens on one specific date even though the availability pattern repeats). Owner manages the weekly pattern by inserting/toggling `active` on rows in `availability_slot` — no per-week maintenance needed.

Deliverer-facing view

A separate, restricted view for whoever's doing pickups/deliveries — usable from a phone, on the road:

- **See client info:** phone number, address, `order.notes` (specifications) — all readable via the queries above.
- **See payment status without the amount:** `order.paid_at` is non-null once paid. That column (and `status`) is readable by the anon key; `total` is not (see above) — so "is this paid?" is answerable, "how much?" is not, by design.
- **Send a payment reminder** without knowing the amount: call the `send-payment-reminder` Edge Function with just an `orderId` — `POST https://yvymhjtswcneouvvgfbc.supabase.co/functions/v1/send-payment-reminder` `Content-Type: application/json { "orderId": "...", "channel": "sms" }` // `channel: "email" | "sms" | "both"`, defaults to both
The function looks up the balance and customer contact info server-side and sends the reminder — the caller never sees the amount. It's a no-op (200, `{"skipped": "already paid"}`) if the order is already paid, so it's safe to expose as a plain button with no extra logic needed on your end.
- **Automatic SMS on initial payment:** already wired — when an order's `status` becomes `DEPOSIT_PAID`, a database trigger fires an SMS to the customer's phone (via Twilio) in addition to the existing email. Nothing for you to build here; just know it happens.

Both notification functions (`send-order-update` , `send-payment-reminder`) are **not live yet** — they need Zoho SMTP and Twilio credentials set as Supabase secrets on the source project. That's the source repo owner's action item, not yours; the endpoints will just silently no-op on the email/SMS leg until those are set.

Open question, not yet decided — don't build against this

Owner is considering turning the admin dashboard into a native app (Apple Developer account already purchased) instead of a page on leshoeshop.ca. **No decision has been made.** If this comes up, don't assume a direction — check with the source repo owner first. Building the web version per this brief is the right move either way (a native app would likely wrap or call the same Supabase backend).

Known gaps (already flagged, not yours to solve silently)

- `stock_item` and `order / customer` are empty — dashboard will show real structure but no data until orders start flowing and par levels are entered.
- No zone/route field for deliveries.
- No auth yet.
- Pickup-fee distance calculation is currently wrong (not anchored on the shop's real address) — needs fixing, not just documenting.

If anything here doesn't match what you find in the live database (schema drift, a renamed column, etc.), trust the live database and flag the mismatch — this doc describes it as of when it was written.